

Additional Core Language Working Group "ready" and "tentatively ready" Issues for the February, 2017 (Kona) meeting

Section references in this document reflect the section numbering of document [WG21 N4606](#).

426. Identically-named variables, one internally and one externally linked, allowed?

Section: 3.5 [basic.link] **Status:** ready **Submitter:** Steve Adamczyk **Date:** 2 July 2003

An example in 3.5 [basic.link] paragraph 6 creates two file-scope variables with the same name, one with internal linkage and one with external.

```
static void f();
static int i = 0;           //1
void g() {
    extern void f();        // internal linkage
    int i;                 //2: i has no linkage
    {
        extern void f();    // internal linkage
        extern int i;       //3: external linkage
    }
}
```

Is this really what we want? C99 has 6.2.2.7/7, which gives undefined behavior for having an identifier appear with internal and external linkage in the same translation unit. C++ doesn't seem to have an equivalent.

Notes from October 2003 meeting:

We agree that this is an error. We propose to leave the example but change the comment to indicate that line //3 has undefined behavior, and elsewhere add a normative rule giving such a case undefined behavior.

Proposed resolution (October, 2005) [SUPERSEDED]:

Change 3.5 [basic.link] paragraph 6 as indicated:

...Otherwise, if no matching entity is found, the block scope entity receives external linkage. **If, within a translation unit, the same entity is declared with both internal and external linkage, the behavior is undefined.**

[Example:

```
static void f();
static int i = 0;           // 1
void g () {
    extern void f ();       // internal linkage
    int i;                 // 2: i has no linkage
    {
        extern void f ();   // internal linkage
        extern int i;       // 3: external linkage
    }
}
```

There are three objects named `i` in this program. The object with internal linkage introduced by the declaration in global scope (line //1), the object with automatic storage duration and no linkage introduced by the declaration on line //2, and the object with static storage duration and external linkage introduced by the declaration on line //3. **Without the declaration at line //2, the declaration at line //3 would link with the declaration at line //1. But because the declaration with internal linkage is hidden, //3 is given external linkage, resulting in a linkage**

conflict. —end example]

Notes from the April 2006 meeting:

According to 3.5 [basic.link] paragraph 9, the two variables with linkage in the proposed example are not “the same entity” because they do not have the same linkage. Some other formulation will be needed to describe the relationship between those two variables.

Notes from the October 2006 meeting:

The CWG decided that it would be better to make a program with this kind of linkage mismatch ill-formed instead of having undefined behavior.

Proposed resolution (November, 2016):

Change 3.5 [basic.link] paragraph 6 as follows:

...Otherwise, if no matching entity is found, the block scope entity receives external linkage. **If, within a translation unit, the same entity is declared with both internal and external linkage, the program is ill-formed.** [Example:

```
static void f();
static int i = 0;    // #1
void g() {
    extern void f();    // internal linkage
    int i;              // #2: i has no linkage
    {
        extern void f(); // internal linkage
        extern int i;    // #3 external linkage, ill-formed
    }
}
```

There are three objects named `i` in this program. The object with internal linkage introduced by the declaration in global scope (line #1), the object with automatic storage duration and no linkage introduced by the declaration on line #2, and the object with static storage duration and external linkage introduced by the declaration on line #3.

Without the declaration at line #2, the declaration at line #3 would link with the declaration at line #1. Because the declaration with internal linkage is hidden, however, #3 is given external linkage, making the program ill-formed. —end example]

727. In-class explicit specializations

Section: 14.7.3 [temp.expl.spec] **Status:** ready **Submitter:** Faisal Vali **Date:** 5 October, 2008

14.7.3 [temp.expl.spec] paragraph 2 requires that explicit specializations of member templates be declared in namespace scope, not in the class definition. This restriction does not apply to partial specializations of member templates; that is,

```
struct A {
    template<class T> struct B;
    template <class T> struct B<T*> { }; // well-formed
    template <> struct B<int*> { }; // ill-formed
};
```

There does not seem to be a good reason for this inconsistency.

Additional note (October, 2013):

EWG has requested CWG to consider resolving this issue. See EWG issue 41.

Additional note, November, 2014:

See also paper N4090.

Proposed resolution (March, 2017):

1. Change 14.5.5 [temp.class.spec] paragraph 5 as follows:

A class template partial specialization may be declared **or redeclared** in any **namespace** scope in which the

corresponding primary template may be defined (7.3.1.2 [namespace.memdef] and , 9.2 [class.mem], 14.5.2 [temp.mem]). [Example:

```
template<class T> struct A {
    struct C {
        template<class T2> struct B { };
        template<class T2> struct B<T2**> { }; // partial specialization #1
    };
};

// partial specialization of A<T>::C::B<T2>
template<class T> template<class T2>
    struct A<T>::C::B<T2*> { }; // #2

A<short>::C::B<int*> absip; // uses partial specialization #2
```

—end example]

2. Change 14.7.3 [temp.expl.spec] paragraph 2 as follows:

An explicit specialization shall be declared in a namespace enclosing the specialized template. An explicit specialization whose *declarator-id* or *class-head-name* is not qualified shall be declared in the nearest enclosing namespace of the template, or, if the namespace is inline (7.3.1 [namespace.def]), any namespace from its enclosing namespace set. Such a declaration may also be a definition may be declared in any scope in which the corresponding primary template may be defined (7.3.1.2 [namespace.memdef], 9.2 [class.mem], 14.5.2 [temp.mem]). If the declaration is not a definition, the specialization may be defined later (7.3.1.2 [namespace.memdef]).

1622. Empty aggregate initializer for union

Section: 8.6.1 [dcl.init.aggr] **Status:** tentatively ready **Submitter:** Daveed Vandevoorde **Date:** 2013-02-14

According to 8.6.1 [dcl.init.aggr] paragraph 15,

When a union is initialized with a brace-enclosed initializer, the braces shall only contain an *initializer-clause* for the first non-static data member of the union.

This would appear to preclude using {} as the initializer for a union, which would otherwise have reasonable semantics. Is there a reason for this restriction?

Also, paragraph 7 reads,

If there are fewer *initializer-clauses* in the list than there are members in the aggregate, then each member not explicitly initialized shall be initialized from an empty initializer list (8.6.4 [dcl.init.list]).

There should presumably be special treatment for unions, so that only a single member is initialized in such cases.

(See also issue 1460.)

Proposed resolution (November, 2016) [SUPERSEDED]:

Change 8.6.1 [dcl.init.aggr] paragraph 8 as follows:

If there are fewer *initializer-clauses* in the list than there are elements in the aggregate, then each **Each non-variant element of the aggregate that is** not explicitly initialized **shall be is** initialized from its default member initializer (9.2 [class.mem]) or, if there is no default member initializer, **copy-initialized** from an empty initializer list (8.6.4 [dcl.init.list]). **If the aggregate is a union and the initializer list is empty, then**

- if any union member has a default member initilizer, that member is initialized from its default member initializer;**
- otherwise, the first member of the union (if any) is copy-initialized from an empty initializer list.**

[Example:...

Proposed resolution (February, 2017):

The resolution of issue 2272 also resolves this issue.

1710. Missing `template` keyword in *class-or-decltype*

Section: 10 [class.derived] **Status:** ready **Submitter:** Richard Smith **Date:** 2013-07-03

A *class-or-decltype* is used as a *base-specifier* and as a *mem-initializer-id* that names a base class. It is specified in 10 [class.derived] paragraph 1 as:

```
class-or-decltype:
    nested-name-specifieropt class-name
    decltype-specifier
```

Consequently, a declaration like

```
template<typename T> struct D : T::template B<int>::template C<int> {};
```

is ill-formed, although most implementations accept it; some actually require the use of the `template` keyword, although the relevant wording in 14.2 [temp.names] paragraph 4 only requires it in a *qualified-id*, not in a *class-or-decltype*. It would probably be good to add a production like

```
nested-name-specifier template simple-template-id
```

to the definition of *class-or-decltype* and explicitly mention those contexts in 14.2 [temp.names] as not requiring use of the `template` keyword.

Additional note (January, 2014):

This is effectively issues 314 and 343.

See also issue 1812.

Proposed resolution (February, 2014) [SUPERSEDED]:

1. Change 9 [class] paragraph 3 as follows:

If a class is marked with the *class-virt-specifier* `final` and it appears as a *base-type-specifier* *class-or-decltype* in a *base-clause* (Clause 10 [class.derived]), the program is ill-formed. Whenever a *class-key* is followed...

2. Change the grammar in 10 [class.derived] paragraph 1 as follows:

```
base-specifier:
    attribute-specifier-seqopt base-type-specifier class-or-decltype
    attribute-specifier-seqopt virtual access-specifieropt base-type-specifier class-or-decltype
    attribute-specifier-seqopt access-specifier virtualopt base-type-specifier class-or-decltype
class-or-decltype:
    nested-name-specifieropt class-name
    nested-name-specifier template simple-template-id
    decltype-specifier
base-type-specifier:
    class-or-decltype
access-specifier:
    ...
```

3. Delete paragraph 4 and change paragraph 5 of 14.2 [temp.names] as follows, splitting paragraph 5 into two paragraphs and moving the example from paragraph 4 into paragraph 5:

When the name of a member template specialization appears after `·` or `->` in a *postfix-expression* or after a *nested-name-specifier* in a *qualified-id*, and the object expression of the *postfix-expression* is type-dependent or the *nested-name-specifier* in the *qualified-id* refers to a dependent type, but the name is not a member of the current instantiation (14.6.2.1 [temp.dep.type]), the member template name must be prefixed by the keyword `template`. Otherwise the name is assumed to name a non-template. [Example: ... —end example]

A name prefixed by the keyword `template` shall be a *template-id* or the name shall refer to a class template **or alias template**. [Note: The keyword `template` may not be applied to non-template members of class templates. —end note] **The nested-name-specifier (N4567_5.1.1 [expr.prim.general]) of**

- **a class-head-name (Clause 9 [class]) or enum-head (7.2 [dcl.enum]) (if any) or**
- **a qualified-id in a declarator-id (Clause 8 [dcl.decl]),**

or a nested-name-specifier directly contained in such a nested-name-specifier (recursively), shall not be of the form

nested-name-specifier `template` simple-template-id ::

[Note: That is, a *simple-template-id* shall not be prefixed by the keyword `template` in these cases. —end note]

The keyword `template` is optional in a *typename-specifier* (14.6 [temp.res]), *elaborated-type-specifier* (7.1.7.3 [dcl.type.elab]), *using-declaration* (7.3.3 [namespace.udecl]), or *class-or-decltype* (Clause 10 [class.derived]), and in recursively directly-contained *nested-name-specifiers* thereof. In these contexts, a `<` token is always assumed to introduce a *template-argument-list*. [Note: Thus, if the preceding name is not a *template-name*, the program is ill-formed. —end note] In other contexts, when the name of a member template specialization appears after a *nested-name-specifier* that denotes a dependent type, but the name is not a member of the current instantiation, the member template name shall be prefixed by the keyword `template`. Similarly, when the name of a member template specialization appears after `.` or `->` in a *postfix-expression* (5.2 [expr.post]) and the object expression of the *postfix-expression* is type-dependent, but the name is not a member of the current instantiation (14.6.2.1 [temp.dep.type]), the member template name shall be prefixed by the keyword `template`. Otherwise, the name is assumed to name a non-template. [Example:

<From original paragraph 4>

—end example] [Note: As is the case with the *typename* prefix...

This resolution also resolves issues 314, 343, 1794, and 1812.

Additional note, November, 2014:

Concerns have been expressed over the clarity and organization of the proposed resolution, so the issue has been moved back to "review" status to allow CWG to address these concerns.

Proposed resolution, March, 2017:

1. Change 9 [class] paragraph 3 as follows:

If a class is marked with the *class-virt-specifier* `final` and it appears as a *base-type-specifier* **class-or-decltype** in a *base-clause* (Clause 10 [class.derived]), the program is ill-formed. Whenever a *class-key* is followed by a *class-head-name*...

2. Change the grammar in 10 [class.derived] paragraph 1 as follows:

base-specifier:

*attribute-specifier-seq*_{opt} *base-type-specifier* **class-or-decltype**
*attribute-specifier-seq*_{opt} *virtual access-specifier*_{opt} *base-type-specifier* **class-or-decltype**
*attribute-specifier-seq*_{opt} *access-specifier* *virtual*_{opt} *base-type-specifier* **class-or-decltype**

class-or-decltype:

*nested-name-specifier*_{opt} *class-name*
nested-name-specifier `template` simple-template-id
decltype-specifier

base-type-specifier:
class-or-decltype

3. Change 10 [class.derived] paragraph 2 as followx:

The type denoted by a *base-type-specifier* **A class-or-decltype** shall **be denote** a class type that is not an incompletely defined class (Clause 9 [class]); **this. The class denoted by the class-or-decltype of a base-**

specifier is called a *direct base class* for the class being defined. During the lookup for a base class name...

4. Change 14.2 [temp.names] paragraphs 4 and 5 as follows:

When the name of a member template specialization appears after `.or->` in a *postfix-expression* or after a *nested-name-specifier* in a *qualified-id*, and the object expression of the *postfix-expression* is type-dependent or the *nested-name-specifier* in the *qualified-id* refers to a dependent type, but the name is not a member of the current instantiation. The keyword `template` is said to appear at the top level in a *qualified-id* if it appears outside of a *template-argument-list* or *decltype-specifier*. In a *qualified-id* of a *declarator-id* or in a *qualified-id* formed by a *class-head-name* (Clause 9 [class]) or *enum-head-name* (7.2 [dcl.enum]), the keyword `template` shall not appear at the top level. In a *qualified-id* used as the name in a *typename-specifier* (14.6 [temp.res]), *elaborated-type-specifier* (7.1.7.3 [dcl.type.elab]), *using-declaration* (7.3.3 [namespace.udecl]), or *class-or-decltype* (Clause 10 [class.derived]), an optional keyword `template` appearing at the top level is ignored. In these contexts, a `<` token is always assumed to introduce a *template-argument-list*. In all other contexts, when naming a template specialization of a member of an **unknown specialization** (14.6.2.1 [temp.dep.type]), the member template name **must** **shall** be prefixed by the keyword `template`. ~~Otherwise the name is assumed to name a non-template.~~ [Example:...

A name prefixed by the keyword `template` shall be a *template-id* or the name shall refer to a class template **or an alias template**. [Note: The keyword `template` may not be applied to non-template members of class templates. —end note]...

5. Change 14.6 [temp.res] paragraph 5 as follows:

A qualified name used as the name in a ~~mem-initializer-id, a base-specifier,~~ **class-or-decltype (Clause 10 [class.derived])** or an *elaborated-type-specifier* is implicitly assumed to name a type, without the use of the `typename` keyword. In a *nested-name-specifier* that immediately contains a *nested-name-specifier* that depends on a template parameter, the identifier or *simple-template-id* is implicitly assumed to name a type, without the use of the `typename` keyword. [Note: The `typename` keyword is not permitted by the syntax of these constructs. —end note]

2196. Zero-initialization with virtual base classes

Section: 8.6 [dcl.init] **Status:** ready **Submitter:** Richard Smith **Date:** 2015-11-06

Proposed resolution (November, 2016):

Change 8.6 [dcl.init] bullet 6.2 as follows:

To *zero-initialize* an object or reference of type τ means:

- if τ is a scalar type (3.9 [basic.types]), the object is initialized to the value obtained by converting the integer literal 0 (zero) to τ ;¹⁰³
- if τ is a (possibly cv-qualified) non-union class type, each non-static data member, **each non-virtual base class subobject**, and, **if the object is not a base class subobject**, each **virtual base-** class subobject is zero-initialized and padding is initialized to zero bits;
- ...

2198. Linkage of enumerators

Section: 3.5 [basic.link] **Status:** ready **Submitter:** Richard Smith **Date:** 2015-11-12

According to the rules in 3.5 [basic.link] paragraph 4, the enumerators of an enumeration type with linkage also have linkage. Having same-named enumerators in different translation units would seem to be innocuous. Is there a rationale for this rule?

Proposed resolution (March, 2017):

1. Delete 3.5 [basic.link] bullet 4.5:

An unnamed namespace or a namespace declared directly or indirectly within an unnamed namespace has internal linkage. All other namespaces have external linkage. A name having namespace scope that has not been given internal linkage above has the same linkage as the enclosing namespace if it is the name of

- ...
- **an enumerator belonging to an enumeration with linkage; or**
- ...

2. Change 3.5 [basic.link] paragraph 9 as follows:

Two names that are the same (Clause 3 [basic]) and that are declared in different scopes shall denote the same variable, function, type, **enumerator**, template or namespace if...

2211. Hiding by lambda captures and parameters

Section: 5.1.5 [expr.prim.lambda] **Status:** tentatively ready **Submitter:** Ville Voutilainen **Date:** 2015-12-07

Proposed resolution (February, 2017):

1. Add the following as a new paragraph after 5.1.5 [expr.prim.lambda] paragraph 11:

The *identifier* in a *simple-capture* is looked up using the usual rules for unqualified name lookup (3.4.1 [basic.lookup.unqual]); each such lookup shall find an entity. An entity that is designated by a *simple-capture* is said to be *explicitly captured*, and shall be **this* (when the *simple-capture* is “*this*” or “** this*”) or a variable with automatic storage duration declared in the reaching scope of the local lambda expression.

If an *identifier* in a *simple-capture* appears as the *declarator-id* of a parameter of the *lambda-declarator's parameter-declaration-clause*, the program is ill-formed. [Example:

```
void f() {  
    int x = 0;  
    auto g = [x](int x) { return 0; } // error: parameter and simple-capture have the same name  
}
```

—end example]

2. Change the example of 5.1.5 [expr.prim.lambda] paragraph 12 as follows:

```
int x = 4;  
auto y = [&r = x, x = x+1]()->int {  
    r += 2;  
    return x+2;  
}(); // Updates : : x to 6, and initializes y to 7.  
auto z = [a = 42](int a) { return 1; } // error: parameter and local variable have the same name
```

2247. Lambda capture and variable argument list

Section: 5.1.5 [expr.prim.lambda] **Status:** tentatively ready **Submitter:** Aaron Ballman **Date:** 2016-03-11

Notes from the December, 2016 teleconference:

Such examples should have undefined behavior; `va_start` should only be permitted to access the arguments for the current function.

Proposed resolution (February, 2017):

Change 18.10.1 [cstdarg.syn] paragraph 1 as follows:

The contents of the header `<cstdarg>` are the same as the C standard library header `<stdarg.h>`, with the following changes: The restrictions that ISO C places on the second parameter to the `va_start()` macro in header `<stdarg.h>` are different in this International Standard. The parameter `parmN` is **the identifier of** the rightmost parameter in the

variable parameter list of the function definition (the one just before the ...).²²³ **If the parameter `parmN` is a pack expansion (14.5.3 [temp.variadic]) or an entity resulting from a lambda capture (5.1.5 [expr.prim.lambda]), the program is ill-formed, no diagnostic required.** If the parameter `parmN` is of a reference type, or of a type that is not compatible with the type that results when passing an argument for which there is no parameter, the behavior is undefined.

2248. Problems with sized delete

Section: 5.3.5 [expr.delete] **Status:** ready **Submitter:** Richard Smith **Date:** 2016-03-14

Proposed resolution (December, 2016):

Change 5.3.5 [expr.delete] paragraph 11 as follows:

When a *delete-expression* is executed, the selected deallocation function shall be called with the address of the **block of storage to be reclaimed** **most-derived object in the *delete object* case, or the address of the object suitably adjusted for the array allocation overhead (5.3.4 [expr.new]) in the *delete array* case,** as its first argument. If a deallocation function with a parameter of type `std::align_val_t` is used, the alignment of the type of the object to be deleted is passed as the corresponding argument. If a deallocation function with a parameter of type `std::size_t` is used, the size of the **block** **most-derived type, or of the array plus allocation overhead, respectively,** is passed as the corresponding argument. **[Note: If this results in a call to a usual deallocation function, and either the first argument was not the result of a prior call to a usual allocation function or the second argument was not the corresponding argument in said call, the behavior is undefined (18.6.2.1 [new.delete.single], 18.6.2.2 [new.delete.array]). —end note]**

2251. Unreachable enumeration list-initialization

Section: 8.6.4 [dcl.init.list] **Status:** ready **Submitter:** Richard Smith **Date:** 2016-03-22

Proposed resolution (December, 2016):

Reorder the bullets in 8.6.4 [dcl.init.list] paragraph 3 as follows:

List-initialization of an object or reference of type τ is defined as follows:

- ...
 - Otherwise, if τ is a class type, constructors are considered...
 - **Otherwise, if τ is an enumeration with a fixed underlying type (7.2 [dcl.enum]), the *initializer-list* has a single element v , and the initialization is direct-list-initialization, the object is initialized with the value $\tau(v)$ (5.2.3 [expr.type.conv]); if a narrowing conversion is required to convert v to the underlying type of τ , the program is ill-formed. [Example:...**
 - Otherwise, if the initializer list has a single element of type E ...
 - Otherwise, if τ is a reference type...
 - ~~Otherwise, if τ is an enumeration with a fixed underlying type...~~
-

2268. Unions with mutable members in constant expressions revisited

Section: 7.1.5 [dcl.constexpr] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2016-05-26

Proposed resolution (February, 2017):

1. Add the following as a new bullet following 5.20 [expr.const] bullet 2.8

A *conditional-expression* *e* is a *core constant expression* unless the evaluation of *e*, following the rules of the abstract machine (1.9 [intro.execution]), would evaluate one of the following expressions:

- ...
- an lvalue-to-rvalue conversion (4.1 [conv.lval]) that is applied to a glvalue that refers to a non-active member of a union or a subobject thereof;
- **an invocation of an implicitly-defined copy/move constructor or copy/move assignment operator for a union whose active member (if any) is mutable, unless the lifetime of the union object began within the evaluation of *e*;**
- ...

2. Delete bullet 3.2 in 7.1.5 [dcl.constexpr]:

- ~~for a defaulted copy/move assignment, the class of which it is a member shall not have a mutable subobject that is a variant member;~~

3. Delete bullet 4.2 in 7.1.5 [dcl.constexpr]:

- ~~for a defaulted copy/move constructor, the class shall not have a mutable subobject that is a variant member;~~

2272. Implicit initialization of aggregate members of reference type

Section: 8.6.1 [dcl.init.aggr] **Status:** tentatively ready **Submitter:** Vinny Romano **Date:** 2016-06-10

Proposed resolution (February, 2017):

1. Change 8.6.1 [dcl.init.aggr] paragraph 8 as follows:

If there are fewer *initializer-clauses* in the list than there are elements in ~~the~~ **a non-union** aggregate, then each element not explicitly initialized ~~shall be initialized from its default member initializer (9.2 [class.mem]) or, if there is no default member initializer, from an empty initializer list (8.6.4 [dcl.init.list]);~~ **is initialized as follows:**

- **If the element has a default member initializer (9.2 [class.mem]), the element is initialized from that initializer.**
- **Otherwise, if the element is not a reference, the element is copy-initialized from an empty initializer list (8.6.4 [dcl.init.list]).**
- **Otherwise, the program is ill-formed.**

If the aggregate is a union and the initializer list is empty, then

- **if any variant member has a default member initializer, that member is initialized from its default member initializer;**
- **otherwise, the first member of the union (if any) is copy-initialized from an empty initializer list.**

[Example:...

2. Delete 8.6.1 [dcl.init.aggr] paragraph 11:

~~If an incomplete or empty *initializer-list* leaves a member of reference type uninitialized, the program is ill-formed.~~

This resolution also resolves issue 1622.

2276. Dependent `noexcept` and function type-dependence

Section: 14.6.2.3 [temp.dep.constexpr] **Status:** tentatively ready **Submitter:** Maxim Kartashev **Date:** 2016-06-23

Proposed resolution (February, 2017):

Add the following as a new bullet following 14.6.2.1 [temp.dep.type] bullet 9.6:

- ...
- an array type whose element type is dependent or whose bound (if any) is value-dependent,
- **a function type whose exception specification is value-dependent,**
- ...